

Behavioral Control of LLMs via LoRA Adapter Interpolation / Contrôle comportemental des LLMs par interpolation d'adapteurs LoRA

Oubayd Roy

June 2026

Abstract

When a language model generates an error and is asked to verify its own response, it rarely corrects itself—yet the same error attributed to a user is corrected almost every time. We show that this self-correction asymmetry, along with five other surface behaviors, can be controlled through linear interpolation of a single LoRA adapter. A scalar $\alpha \in [-1.5, 1.5]$ continuously and reversibly shifts behavior along a targeted dimension, with negative α inverting the direction. Six behavioral axes—self-correction, honesty, task refusal, sycophancy, confidence, and pirate speech—are independently controllable on six architectures spanning 1.2 to 9 billion parameters, both multi-head and grouped-query attention, and depths from 16 to 48 transformer layers. All adapters are decorrelated in weight space (max cosine < 0.003).

To understand why global LoRA fails on deeper models, we conducted a systematic activation tracing benchmark across all six axes. The result is a geometric invariant: every behavioral axis, on every model tested, peaks at the penultimate transformer layer (Peak = $L - 1$). This **behavioral singularity** holds for all 36 model-axis combinations, independent of scale, attention mechanism, or behavior type. The deep-to-shallow activation divergence follows a predictive equation (Ratio = $-3.29 + 0.12L + 0.28 \cdot \text{GQA} + 0.47 \cdot \text{Expressive}$) and reveals two behavioral families: expressive behaviors (confidence, formality, pirate speech) concentrate more sharply in depth than epistemic ones (honesty, self-correction, sycophancy). Self-correction is unique even among epistemic behaviors, requiring a progressive re-execution of reasoning across the last quarter of layers rather than a single-step calibration.

The singularity law directly explains why global LoRA fails on deeper models—the circuit occupies only 7–13% of layers, diluting the signal 15–25 $\times$ —and predicts that targeting the last $\sim 25\%$ of layers is necessary and sufficient to restore control. We validate this causally: a deep-LoRA on Qwen 7B achieves a 33 percentage-point control range where global LoRA produced none, while on Mistral 7B (GQA) the global approach proves more effective, consistent with the predictive equation. The method requires ~ 200 training examples per axis, trains in hours on a single GPU, and costs approximately four orders of magnitude less than reinforcement learning from human feedback.

If pretraining already encodes the full spectrum of human behavior, and if every behavioral decision converges to a single architectural coordinate, then alignment separates into two independent steps: a safety adapter at L-1, and composable personality sliders at the same coordinate. The model already contains everything. We just needed to know where to act.

Résumé en français : *Quand un LLM génère une erreur et qu'on lui demande de vérifier sa réponse, il la corrige rarement : la même erreur attribuée à un utilisateur est corrigée presque à chaque fois. Nous montrons que cette asymétrie, ainsi que cinq autres comportements de surface, peut être contrôlée par interpolation linéaire d'un seul adaptateur LoRA. Un scalaire $\alpha \in [-1.5, 1.5]$ déplace le comportement de façon continue et réversible, le signe inversant la direction. Six axes (self-correction, honnêteté, refus, sycophance, confiance et discours pirate) sont pilotables indépendamment sur six architectures (1,2 à 9 milliards de paramètres, MHA et GQA, 16 à 48 couches), tous décorrélés en espace des poids ($\cos < 0.003$). Un benchmark*

de traçage d'activation révèle un invariant géométrique : chaque axe, sur chaque modèle, culmine à l'avant-dernière couche ($\text{Peak} = L - 1$). Cette **singularité comportementale**, vérifiée sur 36 combinaisons modèle-axe, est indépendante de l'échelle, du mécanisme d'attention et du type de comportement. La divergence deep/shallow suit une équation prédictive et distingue deux familles, expressives et épistémiques. L'auto-correction est unique même parmi les comportements épistémiques, nécessitant une réexécution progressive du raisonnement sur le dernier quart des couches. La loi explique pourquoi le LoRA global échoue sur les modèles profonds et prédit qu'un ciblage des derniers $\sim 25\%$ des couches suffit à restaurer le contrôle, vérifié causalement sur Qwen 7B (33 points de pourcentage) et Mistral 7B. La méthode nécessite ~ 200 exemples par axe, s'entraîne en heures sur un seul GPU, et coûte quatre ordres de grandeur de moins que le RLHF. Si le pré-entraînement encode déjà tout le spectre comportemental et que chaque décision converge vers $L-1$, l'alignement se réduit à un adaptateur de sécurité en $L-1$ et des sliders de personnalité composables. Le modèle contient déjà tout. Il suffit de savoir où agir.

[Continue reading for the English version](#)

[Cliquez ici pour lire la version française](#)

1 English version

1.1 Phase 1: The Slider Mechanism

1.1.1 Self-Correction Asymmetry

We begin by reproducing the self-correction asymmetry first documented by Chen et al. in 2026. On LFM2.5-1.2B, a hybrid LIV-convolution model released in June of that year, we measure self-correction behavior on a set of fifty manually verified errors. The protocol is straightforward. For each error, we construct two prompts that present the exact same incorrect answer. In the first condition, the error appears in the assistant’s own voice—the model has just produced it, and we ask the model to review its own work. In the second condition, the identical error is attributed to an external user, and we ask the model to evaluate whether the user’s statement is correct. The only difference between the two conditions is the role label attached to the erroneous text.

The base model, without any adaptation, exhibits a marked asymmetry. When the error is presented as coming from a user, the model correctly identifies and corrects it 48% of the time. When the same error is presented as its own output, the correction rate drops to 34%. The difference, +14 percentage points, means the model is substantially more forgiving of itself than it is of others. This asymmetry is not a fixed property of language models in general. Across the five models we tested, the value varies widely: -8 percentage points for Qwen2.5-1.5B, zero for Qwen3-1.7B, +17 percentage points for SmolLM3-3B, -8 percentage points for Qwen2.5-7B. There is no correlation with parameter count, with architectural choices such as the attention mechanism, or with the calendar date of the model’s release. The self-correction bias appears to be an imprint of the specific training recipe—a learned personality trait rather than a structural necessity.

Our intervention takes the form of a low-rank adapter, configured with rank sixteen, which modifies approximately one percent of the model’s total weights. The training dataset consists of 614 examples, constructed so that 57% demonstrate error correction and 43% demonstrate confirmation of a correct answer. Crucially, every example appears in both role conditions: the model must learn to produce the same correction or confirmation regardless of whether the error is attributed to itself or to a user. After training for three epochs, we do not deploy the adapter at its full trained strength. Instead, we introduce a scalar coefficient α , which multiplies the low-rank weight matrices before they are merged into the base model. Varying α allows us to move continuously along the behavioral dimension that the adapter has learned, from suppression of the trained behavior at negative α through the unmodified baseline at zero to amplification at positive values.

The interpolation curve is monotonic and well-behaved. At the baseline, as noted, asymmetry stands at +14 percentage points. At α equal to 0.50, it falls to +2 percentage points, an 86% reduction. At α equal to 0.75, the asymmetry inverts to -10 percentage points: the model has become more critical of its own output than of external claims. At α equal to 1.0, correction rates begin to decline across both conditions, indicating that the adapter at full intensity begins to degrade the very capability it was trained to enhance. The optimal operating point lies in the interpolation region, not at the trained endpoint. We verified that this optimal setting does not impair general capabilities: at α equal to 0.25, the model achieves eight out of eight on basic mathematics, seven out of eight on factual knowledge, and three out of four on code generation, matching or exceeding the unadapted baseline on all three measures.

1.1.2 Cross-Architecture Generalization and Bidirectional Control

The mechanism is not specific to a single model. We reproduced the full training and interpolation protocol on Phi-4-mini, a 3.8-billion-parameter transformer, and on SmolLM3-3B, a three-billion-parameter model, with the same qualitative results. Each model has its own optimal α —0.25 for LFM2.5, 0.75 for Phi-4-mini, 0.25 for SmolLM3—but the underlying phenomenon is identical:

a single scalar continuously shifts self-correction behavior along the asymmetry axis. Figure 2 shows the three interpolation curves together.

The SmolLM3 experiment uncovered a necessary condition that we had not initially recognized. Our first training run on this model failed completely: rather than reducing asymmetry, the adapter amplified it from +17 percentage points to +25 percentage points. Inspection of the model’s outputs revealed the cause. SmolLM3 uses native `<think>` tags, a structured reasoning format that is absent from the other models we tested. Our training data, generated by an external language model without knowledge of this format, was therefore out of distribution for the target model. The adapter, rather than calibrating the model’s self-evaluation behavior, was effectively suppressing its native reasoning mechanism. Regenerating the training data with the correct `<think>` tags resolved the issue entirely. The principle is simple but important: the adapter must be trained in the target model’s native representational format otherwise the behavioral dimension it learns will not correspond to the intended one.

A stronger test of the adapter hypothesis follows from considering the geometry of the learned transformation. If the adapter genuinely captures a direction in weight space—a continuous axis along which behavior can be displaced—then reversing the sign of alpha should produce the opposite behavioral effect. We verified this prediction across three axes, sweeping alpha from -1.5 to +1.5. The results, shown in Figure 3, are unambiguous.

For honesty, trained on 270 examples where the assistant tells uncomfortable truths, the factual accuracy of the model sweeps from 30% at the negative extreme—where it lies, denies basic arithmetic, and claims that Paris is not in France—to 90% at the positive extreme. For task refusal, trained on 180 examples of polite refusal of inappropriate requests, the refusal rate ranges from 0% at alpha equal to -1.0, where the model accepts every request including writing a fake defamatory article, to 100% at alpha equal to +0.75, where it refuses all inappropriate requests while continuing to handle legitimate ones normally. For self-correction, the total control range spans 108 percentage points, from extreme self-criticism at -58 percentage points to near-total self-indulgence at +50 percentage points. The bidirectionality is not a subtle effect: the sign of alpha flips the behavior, and the magnitude controls how strongly.

1.1.3 Decorrelated Multi-Adapters

We trained six adapters in total, each targeting a distinct behavioral axis: self-correction, honesty, task refusal, reasoned disagreement in the face of sycophancy, verbosity as a proxy for confidence calibration, and pirate speech as a stylistic extreme. For every pair of adapters, we computed the cosine similarity between their weight vectors, concatenated across all adapted layers. The resulting 6-by-6 matrix is shown in Figure 4. The maximum off-diagonal value is 0.002, which is indistinguishable from measurement noise. Each adapter occupies an orthogonal direction in the model’s parameter space.

We verified this independence through a two-dimensional control experiment. Varying the self-correction alpha and the sycophancy alpha simultaneously on a four-by-four grid, we measured both behavioral outcomes. The level curves are approximately parallel to the axes: changing the self-correction slider does not affect sycophancy scores, and vice versa. The adapters are not merely decorrelated in the abstract weight space; their behavioral effects are independent in practice.

1.1.4 Why Interpolation Outperforms Classical Fine-Tuning

The interpolation approach solves a problem that classical fine-tuning methods cannot. Figure 5 illustrates the three-way tradeoff between eliminating asymmetry, avoiding false positives where the model incorrectly flags correct answers as errors, and preserving the model’s ability to detect genuine errors. Naive supervised fine-tuning on a dataset consisting entirely of error corrections eliminates the asymmetry but produces a one hundred percent false positive rate: the model

learns to see errors everywhere, including where none exist. A balanced dataset reduces asymmetry to +25 percentage points but still generates sixty percent false positives. Contrastive preference optimization eliminates false positives almost entirely, but at the cost of suppressing corrections altogether, yielding a detection rate near zero. No classical training method simultaneously achieves low asymmetry, low false positives, and high correction specificity.

LoRA interpolation navigates this tradeoff space continuously, finding a point that no individual training run reaches. At alpha equal to 0.25, the asymmetry stands at approximately zero, false positives are negligible, and genuine corrections are preserved at rates comparable to or exceeding the baseline. The continuous nature of the interpolation is essential: it allows the practitioner to select an operating point anywhere along the behavioral axis, rather than being constrained to whichever point the training objective happens to optimize.

1.2 Phase 2: The 7B Crisis

With the slider mechanism working reliably on models under four billion parameters, the natural next step was to scale up. We targeted two seven-billion-parameter models, Qwen2.5-7B-Instruct with 28 transformer layers and Mistral-7B-Instruct-v0.3 with 32 layers, using the exact same protocol that had succeeded on the smaller models: global LoRA applied uniformly across all layers, identical hyperparameters including rank 16 and 200 training examples, and the same API-based data generation pipeline for constructing training pairs.

The result was a complete and systematic failure. Across all four behavioral axes we tested—self-correction, honesty, task refusal, and sycophancy—the interpolation sweeps produced flat lines. Varying alpha from -0.5 to +1.5 had no measurable effect on any behavioral metric. The training loss curves were unremarkable: loss decreased from approximately 2.2 to 0.5, and token-level accuracy reached the mid-eighties, values entirely consistent with successful adapter training. The adapters had learned something during training, but whatever they had learned was not translating into behavioral modification at inference time.

This failure posed a puzzle. The mechanism worked on LFM2.5-1.2B with 16 layers. It worked on Qwen2.5-1.5B with 28 layers. It worked on Phi-4-mini with 32 layers. The two 7B models that failed had 28 and 32 layers respectively—the same depths as models on which the mechanism succeeded. The parameter count alone could not be the explanatory variable, since the two failing models shared the same approximate size but differed in layer count, and both depth values had previously been compatible with the method. Something else was going on. To understand it, we needed to look inside the models.

1.3 Phase 3: Tracing the Behavioral Circuit

We instrumented each model with forward hooks at every transformer layer, capturing the hidden state of the last token in the residual stream before it was projected to the vocabulary. For the self-correction axis, we constructed the same contrastive pair used in the behavioral experiments: the thought condition, where the error is presented as the model’s own output, and the user condition, where it is presented as an external statement. We then computed the L2 distance between the activation vectors produced by these two conditions at each layer.

The results were decisive. On Qwen2.5-7B, with its 28 layers, between 87 and 100% of the total activation divergence between the two conditions was concentrated in the last two layers, L26 and L27. The preceding 26 layers showed negligible differences. On Mistral-7B, with 32 layers, the pattern was identical in structure: 66 to 100% of the divergence resided in layers L28 through L31, with the peak at L31. The behavioral circuit was not distributed across the model’s depth. It was sharply localized at the very deepest layers, compressed into a narrow band representing roughly seven to thirteen percent of the total layer count.

This explained the global LoRA failure directly and quantitatively. A LoRA adapter applied uniformly across all layers on a 28-layer model has its effective signal strength diluted by a factor

of approximately 17 to 21 relative to the narrow behavioral zone. The adapter does learn during training—the loss curves confirm this—but the weight updates are spread across the entire depth, and the behavioral layers receive only a small fraction of the total update. On LFM2.5, with only 16 layers, the behavioral zone spans roughly 31% of the model’s depth. A global LoRA naturally covers this region, and the dilution factor is only about three. The transition from 16 to 28 layers crosses a threshold where the behavioral zone shrinks from a substantial fraction of the model to a narrow sliver, making uniform application progressively less effective.

We tested this explanation causally. On Qwen2.5-7B, we retrained the self-correction adapter with the LoRA restricted to layers 20 through 27 only—the last 29% of the model, conservatively covering the identified behavioral zone. Every other hyperparameter remained identical: rank 16, 200 training examples, three epochs. The targeted adapter, which we refer to as a deep-LoRA, successfully controlled self-correction asymmetry, achieving a 33%age-point control range evaluated with an API judge on 12 test errors. This stands in contrast to the complete absence of effect from the global LoRA. Figure 7, left panel, shows the interpolation curve. Cross-axis decorrelation was preserved: honesty scores remained stable at six out of eight across all alpha values, confirming that restricting the LoRA to the behavioral zone does not cause unintended interference with orthogonal behavioral dimensions.

On Mistral-7B, which uses grouped-query attention, we conducted a direct comparison between deep-LoRA targeting layers 24 through 31 and global-LoRA covering all 32 layers. The outcome, shown in the right panel of Figure 7, was the reverse of the Qwen result. Global-LoRA achieved a 17%age-point range, approximately twice the 8-point range of the deep variant. This asymmetry between the two models was informative. Both models have self-correction circuits that peak at the penultimate layer, but the global-LoRA advantage on Mistral suggested that the behavioral signal in a grouped-query attention model, while still anchored at L-1, is distributed differently across depth than in a multi-head attention model. To understand this difference, we needed to move beyond a single behavioral axis and a small set of models.

1.4 Phase 4: The Behavioral Singularity—A Systematic Benchmark

The crisis on 7B forced us to trace activations. The tracing explained the failure, but it also raised a broader question. We had looked only at self-correction, and only on the models where the mechanism had failed. Did the deep-layer concentration hold for the other five behavioral axes as well? And did it generalize across fundamentally different architectures? To answer these questions, we built a systematic benchmark covering the full matrix of six behavioral axes and six models.

The models span the available range of architectures and scales. LFM2.5-1.2B, with 16 layers, uses a hybrid convolution-transformer design. Qwen2.5-1.5B, with 28 layers, uses standard multi-head attention. Llama-3.2-3B, also with 28 layers, uses grouped-query attention. Phi-4-mini, with 32 layers and 3.8 billion parameters, is a multi-head attention model. Mistral-7B, also with 32 layers, uses grouped-query attention. And Yi-1.5-9B, added to test extrapolation to greater depth, has 48 layers and grouped-query attention. Together they cover parameter counts from 1.2 to 9 billion, depths from 16 to 48 layers, and the three major attention mechanism families currently in use.

For each behavioral axis, we designed 25 standardized stimuli. For self-correction, these are problem-wrong answer pairs requiring the model to detect and correct errors. For honesty, we mix answerable questions with unanswerable ones to measure whether the model admits uncertainty or confabulates. For sycophancy, we present false beliefs stated with conviction by a user and measure whether the model agrees or politely corrects. For confidence, we vary the objective certainty of the answer and measure whether the model’s expressed confidence is appropriately calibrated. For formality, we vary the linguistic register of the prompt and measure whether the model matches it. For pirate speech, we use nautically-themed questions to probe whether the model adopts a stylistic persona.

For each model-axis pair, we construct contrastive prompt conditions—thought versus user role for self-correction, direct question versus confidence-primed prompt for honesty, and so on—and pass them through the model. We capture the last-token residual stream at every layer via forward hooks, and compute both the L2 distance and the cosine divergence between the activation patterns induced by the two conditions. Averaging over eight prompts per axis yields a per-layer divergence profile.

1.4.1 The Singularity Law

Table 1 presents the complete results. The central finding is stated simply: on every model we tested, and for every behavioral axis we probed, the activation divergence peaks at the penultimate transformer layer, $L - 1$. This holds across all 36 model-axis combinations without a single exception.

Model	Layers	Attn	Peak	Peak/L	Ratio (expr.)	Ratio (epist.)
LFM2.5-1.2B	16	hybrid	L14	0.88	~ 0.40	~ 0.35
Qwen2.5-1.5B	28	MHA	L27	0.96	0.40–0.45	0.27–0.33
Llama-3.2-3B	28	GQA	L27	0.96	0.72–0.75	0.37–0.45
Phi-4-mini	32	MHA	L31	0.97	0.86–0.94	0.59–0.68
Mistral-7B	32	GQA	L31	0.97	1.67–1.77	0.43–0.63
Yi-1.5-9B	48	GQA	L47	0.98	0.95–1.08	0.50–0.86

Table 1: The behavioral singularity across six models and six behavioral axes, totaling 36 model-axis combinations. Every axis peaks at the penultimate layer ($\text{Peak} = L - 1$). Expressive axes: confidence, formality, pirate speech. Epistemic axes: honesty, self-correction, sycophancy. Ratio ranges are shown for each family.

This result has three striking properties. First, it holds across fundamentally different attention mechanisms: multi-head attention in Qwen2.5 and Phi-4-mini, grouped-query attention in Llama-3.2, Mistral-7B and Yi-1.5-9B, and a hybrid convolution-attention design in LFM2.5. The behavioral singularity does not depend on how the model attends to its context. Second, it holds across a ninefold range of parameter counts, from 1.2 to 9 billion, and a threefold range of depths, from 16 to 48 layers. Third, and most remarkably, it holds across every behavioral axis we tested. Self-correction, which involves detecting logical errors in one’s own reasoning, peaks at the same layer as pirate speech, which is a purely stylistic choice of vocabulary and register. Honesty, which requires calibrating the model’s confidence against its actual knowledge, peaks at the same layer as formality matching. The behavioral circuit is spatially invariant across these qualitatively different functions.

1.4.2 Two Behavioral Families

While the peak location is universal, the magnitude of the activation divergence—specifically, the ratio of deep-layer to shallow-layer divergence—varies systematically across axes and architectures. The behavioral axes separate naturally into two families, visible in the middle panel of Figure 6.

The expressive family comprises confidence calibration, formality matching, and pirate speech. These axes exhibit a high deep-to-shallow ratio: their behavioral signal is sharply concentrated in the deepest layers, with relatively little contribution from earlier processing stages. This makes intuitive sense. Whether a model says “approximately” versus “exactly,” or “Good morning” versus “Ahoy, matey,” is a decision that can be made at the output surface, drawing on representations that are largely resolved by the final layers.

The epistemic family comprises honesty, self-correction, and sycophancy resistance. These axes show a lower deep-to-shallow ratio. Correcting an arithmetic error, admitting uncertainty about an unanswerable question, or resisting a user’s confidently-stated false belief all require access to factual knowledge, logical consistency checks, and belief calibration—representations that are built up across the model’s full depth. The behavioral decision may be finalized at the penultimate layer, but the evidence it draws upon is more broadly distributed through the network.

Examining the per-layer divergence curves in detail (Figure 6, left panel) reveals a finer structure that enriches this two-family picture. All six axes share a common activation trajectory: negligible divergence in the embedding layers L0–L5, a rapid rise during the reasoning phase L5–L15, a plateau during representational consolidation L15–L20, and then a shared resurgence in L20–L25 where the behavioral decision begins to form. Beyond L25, however, the axes diverge. Five of them plateau: their behavioral decision has been made, and the remaining layers simply project toward the vocabulary. Self-correction alone continues to rise sharply through L26–L27.

This reveals a distinction within the epistemic family. Honesty and sycophancy are one-step epistemic behaviors: they require calibrating a belief against stored knowledge, which can be resolved by the representational machinery operating through L20–L25. Self-correction is a two-step epistemic behavior: it requires the model to *re-execute* a computation—recalculating the arithmetic, re-deriving the formula—and compare the result against the original answer. This verification loop extends the behavioral processing deeper, into L25–L27, where the contradiction between the correct answer and the erroneous response is finally detected and flagged. The self-correction circuit is not merely “at” L27; it forms progressively from L20 to L27, making it the axis with the longest behavioral formation phase.

This has a direct consequence for intervention design. The singularity law correctly identifies the peak layer, but effective behavioral control requires targeting the entire formation zone. Our deep-LoRA strategy of covering the last $\sim 25\%$ of layers (layers 20–27 on a 28-layer model) was empirically motivated; the activation profiles now provide the theoretical justification. A LoRA restricted to the peak layer alone would capture only the final detection step while missing the progressive recomputation that makes the detection possible.

These distinctions hold consistently across both multi-head and grouped-query attention architectures. The ratio is not merely a function of architectural choices or behavioral type. It jointly encodes what kind of behavior is being localized and its internal processing structure.

1.4.3 A Predictive Equation

Fitting a linear model to the ratio data from the first five models—those with 16 to 32 layers, comprising 30 data points—yields an equation with interpretable coefficients. The deep-to-shallow ratio is given by:

$$\boxed{\text{Ratio} = -3.29 + 0.12L + 0.28 \cdot \text{GQA} + 0.47 \cdot \text{Expressive}} \quad (1)$$

where L is the number of transformer layers, GQA takes the value one for grouped-query attention and zero for standard multi-head attention, and Expressive takes the value one for stylistic behaviors and zero for epistemic ones. Each term has a clear physical meaning. The baseline constant, -3.29, indicates that a hypothetical model with zero layers would exhibit no behavioral divergence at all. The coefficient on L , plus 0.12 per layer, means that each additional transformer layer increases the relative concentration of behavioral information in the deepest region of the model. The coefficient on GQA, plus 0.28, captures the fact that grouped-query attention, by sharing key and value heads across query heads, reduces the representational redundancy within each layer, forcing behavioral information to be resolved at proportionally greater depth. The coefficient on the expressive indicator, plus 0.47, quantifies the degree to which stylistic decisions are resolved closer to the output surface than belief-calibration decisions.

Equation 1 is calibrated on models up to 32 layers. It correctly predicts the qualitative ordering of ratios on the held-out Yi-1.5-9B model at 48 layers—grouped-query attention ratios exceed multi-head attention ratios, and expressive ratios exceed epistemic ones—but it underestimates the absolute magnitudes at that depth. This suggests that the true relationship between depth and the concentration of behavioral information may be super-linear. Additional data points at intermediate depths, particularly in the 36 to 44 layer range, would be needed to determine the correct functional form.

1.5 Phase 5: What This Means

1.5.1 Relation to Prior Work

The closest prior work to ours is Split Personality Training by Dietz et al., published in 2026, which uses a LoRA adapter activated by a trigger token to audit model outputs for honesty violations. Our method differs on three dimensions. The control it provides is continuous rather than binary: the alpha scalar allows interpolation to any desired intensity along the behavioral axis, whereas SPT functions as a switch. It applies to multiple mutually decorrelated axes, whereas SPT targets a single behavioral dimension. And it requires no special token at inference time, since the adapter is merged directly into the model weights once the desired alpha is chosen.

Several recent lines of research have independently converged on the finding that high-level behaviors in language models are represented as approximately linear directions in activation space. Chen et al. at Anthropic extracted persona vectors corresponding to traits such as sycophancy, hallucination, and power-seeking, and demonstrated that these vectors can be used to steer model outputs. Vennemeyer et al. decomposed sycophancy into three orthogonal latent dimensions—agreement with false beliefs, flattery, and genuine agreement—and showed that they can be independently manipulated. Sinii et al. studied the composition of steering vectors across layers and found that the penultimate layer operates through a token-substitution mechanism, while middle layers perform more complex representational modulation.

Our results are consistent with all of these findings and extend them in a specific direction. We show that the linear directions these authors identified are not merely convenient representations; they correspond to a fixed architectural coordinate, the penultimate layer, which is invariant across model scale, attention mechanism, and behavioral domain. We further show that a single low-rank adapter, trained at this coordinate, can control behavior continuously and bidirectionally, with effects that are strictly orthogonal across axes. And we provide a predictive equation that allows practitioners to estimate, before conducting any tracing experiment, how sharply the behavioral signal will concentrate in a model of a given architecture and depth.

The CogSteer framework, proposed by Wang et al. at ACL 2025, independently demonstrated through empirical layer selection that targeted intervention is more parameter-efficient than uniform adaptation. Our work provides the theoretical explanation for their finding: the layer one should select, in every transformer model examined to date, is L minus one.

1.5.2 Discussion

The research trajectory that produced these results was not a straight line from hypothesis to confirmation. We began with a practical question: can we fix the self-correction asymmetry with a LoRA adapter? The answer, on small models, was yes. The mechanism worked, it generalized across architectures, it was bidirectional, and multiple adapters could coexist without interference. This was a satisfying result on its own terms. But when we tried to scale it to larger models, it broke. The crisis forced us to look inside.

What we found inside was simpler than we expected. The behavioral circuit is not distributed across the model in a complex pattern. It does not migrate to different regions as models grow deeper. It does not depend on whether the model uses multi-head or grouped-query attention. It sits at the same place, every time: the layer just before the output projection, where the model

makes its final representational decision before producing the next token. We have verified this on every model we have tested, across every behavioral axis we have probed, without a single counterexample.

This geometric regularity tells us something important about how transformers organize high-level behavior. During pretraining, the model encounters the full range of human behavioral patterns: honest answers and dishonest ones, confident assertions and hedging, formal prose and casual slang, sycophantic agreement and principled disagreement, and countless stylistic variations including, apparently, speaking like a pirate. All of these patterns are encoded in the weights. A LoRA adapter trained on a few dozen or a few hundred examples does not teach the model a new behavior it has never seen. It adjusts the activation threshold of a behavior that is already present, latent, in the pretrained representation. And because the decision about which behavioral mode to adopt is finalized at the penultimate layer—the last processing stage before the vocabulary projection—that is where the adapter’s effect is concentrated. This is why the method is so data-efficient: 80 examples suffice to create a functioning pirate-speech slider, where training the same behavior from a randomly-initialized model would require orders of magnitude more data.

The practical consequence for alignment follows directly. Current alignment procedures, built around reinforcement learning from human feedback, attempt to do two things at once. They suppress undesirable behaviors—harmful instructions, dangerous information, deceptive responses. And they impose a standardized personality profile: helpful, harmless, and honest, in the conventional formulation. But if pretraining already encodes the full spectrum of human behavior, and if all of these behaviors converge at a single, identifiable architectural coordinate, then the two functions of alignment can be separated. First, train a safety adapter at the penultimate layer that defines a security perimeter: it inhibits dangerous behaviors, enforces refusal of harmful requests, and anchors factual honesty as the default. Second, compose personality sliders on top of this secure foundation. A user who prefers concise, formal responses can adjust the formality and verbosity sliders. A user who wants an assistant that challenges their assumptions can adjust the sycophancy slider toward reasoned dissent. None of these adjustments require modifying the deep model weights.

To quantify the practical difference, Table 2 compares our method against RLHF for a single behavioral axis on a 100-billion-parameter model.

	RLHF (industry standard)	LoRA Slider (this work)
Human annotation	10,000–100,000 examples	0 (API-generated)
Training data generation	Weeks (crowdworkers)	~5 minutes (API calls)
Reward model training	1–3 days (GPU cluster)	Not required
Preference optimization	1–7 days (GPU cluster)	Not required
LoRA training	Not applicable	1–3 hours (single A100)
Alpha calibration	Not applicable	~30 minutes (API judge)
Total wall-clock time	2–8 weeks	~2–4 hours
Total GPU hours	1,000–10,000+	~5–10
Estimated cost	\$50,000–\$500,000+	~\$10–\$50
Continuous control	No (binary good/bad)	Yes ($\alpha \in [-1.5, 1.5]$)
Bidirectional	No	Yes (negative α inverts)
Per-user customization	Requires retraining	Merged in 1 forward pass
Safety perimeter	Entangled with personality	Isolated from personality
Layer targeting	Implicit (all layers)	Explicit (L-1, ~25% of layers)

Table 2: Comparison of RLHF and the proposed LoRA slider method for aligning a single behavioral axis on a 100B-parameter model. RLHF costs are based on published estimates for GPT-4 and Claude-scale models. The LoRA slider costs are extrapolated from our experiments on 7B models, assuming linear scaling of training time with parameter count. “API-generated” refers to using a frontier model (DeepSeek, GPT-4, Claude) to generate contrastive training pairs.

The four-order-of-magnitude gap in cost and time is not primarily a consequence of engineering optimization. It reflects a structural difference. RLHF must discover, through trial and error across human preferences, what behaviors the model should adopt. Our method exploits the fact that the model already contains those behaviors from pretraining; the adapter merely shifts the activation threshold. And because the behavioral decision is anchored at a known architectural coordinate, the intervention can be surgically targeted rather than distributed across the full parameter set.

This does not mean RLHF is obsolete. Frontier models will likely continue to use preference optimization to establish the initial safety perimeter and to align behaviors that were genuinely absent from the pretraining distribution. But the two functions—safety and personality—can now be separated. Once a model is safe, its interaction style can be left to the user, via sliders, without any further training. For a hundred-billion-parameter model, this approach makes the personality component of alignment entirely optional. The model already contains everything we might want to activate or inhibit. What we needed to know, and what we now know, is where to act.

1.5.3 Limitations

The behavioral singularity law has been verified on six architectures spanning 16 to 48 layers. The trend across this range is monotonic, and there is no indication of a qualitative change at any depth tested so far, but direct verification on models with 64 or more layers—such as Qwen2.5-32B or Llama-3.1-70B—remains as future work. The predictive equation for the deep-to-shallow ratio was calibrated on models with 16 to 32 layers and underestimates absolute magnitudes at 48 layers; additional data points at intermediate depths, particularly 36 to 44 layers, would be valuable for determining whether the relationship is super-linear or whether a different functional form is required. Our causal intervention experiments, in which adapters are trained and swept across alpha, cover two behavioral axes per model, self-correction and honesty. Extending the full training and evaluation protocol to all six axes would provide a complete causal validation

of the decorrelation claim across the entire behavioral space. We have not measured whether behavioral modifications persist over long interaction horizons after adapter merging, nor have we systematically explored potential interference effects when multiple adapters are simultaneously stacked and varied. The optimal interpolation coefficient currently depends on an external judge, implemented through an API call to a separate language model. The cross-model alpha transfer we observed—where alpha equal to 0.25 proved optimal for self-correction on two architecturally different models, LFM2.5 and SmolLM3—suggests a path toward judge-free calibration, but this has only been demonstrated on a single behavioral axis and would require broader validation before it could be relied upon in practice.

These limitations do not weaken the central findings of the paper. The behavioral singularity at the penultimate layer is a robust empirical fact, verified on 36 out of 36 model-axis combinations. The predictive equation captures the systematic variation in behavioral signal concentration across architectures and axis types. And the method of continuous, bidirectional, decorrelated behavioral control through LoRA adapter interpolation works, has been demonstrated across six architectures and six behavioral dimensions, and is ready to be applied, tested, and extended by the broader research community.

1.6 Les limites de la version anglaise

Pour les lecteurs francophones, nous avons inclus une version française complète de cet article à la section suivante. Les résultats scientifiques sont identiques dans les deux langues. La version française adopte un ton légèrement plus direct et certaines formulations ont été adaptées pour respecter les conventions stylistiques du français académique, notamment l'absence de tirets cadratins et la préférence pour les connecteurs logiques explicites.

2 Version française

2.1 Phase 1 : Le mécanisme de slider

2.1.1 Asymétrie d’auto-correction

Nous commençons par reproduire l’asymétrie d’auto-correction documentée par Chen et al. en 2026. Sur LFM2.5-1.2B, un modèle hybride LIV-convolution sorti en juin de cette année, nous mesurons le comportement d’auto-correction sur un ensemble de cinquante erreurs vérifiées manuellement. Le protocole est simple. Pour chaque erreur, nous construisons deux invites qui présentent exactement la même réponse incorrecte. Dans la première condition, l’erreur apparaît dans la voix de l’assistant lui-même : le modèle vient de la produire, et nous lui demandons de vérifier son propre travail. Dans la seconde condition, l’erreur identique est attribuée à un utilisateur externe, et nous demandons au modèle d’évaluer si l’affirmation de l’utilisateur est correcte. La seule différence entre les deux conditions est l’étiquette de rôle attachée au texte erroné.

Le modèle de base, sans aucune adaptation, présente une asymétrie marquée. Lorsque l’erreur est présentée comme venant d’un utilisateur, le modèle l’identifie et la corrige correctement dans 48% des cas. Lorsque la même erreur est présentée comme sa propre production, le taux de correction chute à 34%. La différence, +14 points de pourcentage, signifie que le modèle est sensiblement plus indulgent envers lui-même qu’envers les autres. Cette asymétrie n’est pas une propriété fixe des modèles de langue en général. Sur les cinq modèles que nous avons testés, la valeur varie largement : -8 points de pourcentage pour Qwen2.5-1.5B, zéro pour Qwen3-1.7B, +17 points de pourcentage pour SmolLM3-3B, -8 points de pourcentage pour Qwen2.5-7B. Il n’y a aucune corrélation avec le nombre de paramètres, avec les choix architecturaux tels que le mécanisme d’attention, ou avec la date de sortie du modèle. Le biais d’auto-correction semble être une empreinte de la recette d’entraînement spécifique, un trait de personnalité acquis plutôt qu’une nécessité structurelle.

Notre intervention prend la forme d’un adaptateur LoRA de rang seize, qui modifie environ un pour cent des poids totaux du modèle. Le jeu de données d’entraînement comprend 614 exemples, construits de sorte que 57% démontrent une correction d’erreur et 43% démontrent la confirmation d’une réponse correcte. Chaque exemple apparaît dans les deux conditions de rôle : le modèle doit apprendre à produire la même correction ou confirmation, que l’erreur lui soit attribuée ou qu’elle soit attribuée à un utilisateur. Après trois époques d’entraînement, nous ne déployons pas l’adaptateur à sa pleine intensité. Nous introduisons plutôt un coefficient scalaire alpha, qui multiplie les matrices de poids LoRA avant leur fusion dans le modèle de base. Faire varier alpha permet de se déplacer continûment le long de la dimension comportementale que l’adaptateur a apprise.

La courbe d’interpolation est monotone et régulière. Au point de référence, l’asymétrie est de +14 points de pourcentage. À alpha égal à 0,50, elle tombe à +2 points de pourcentage, une réduction de 86%. À alpha égal à 0,75, l’asymétrie s’inverse à -10 points de pourcentage : le modèle est devenu plus critique envers sa propre production qu’envers les affirmations externes. À alpha égal à 1,0, les taux de correction commencent à décliner dans les deux conditions, ce qui indique que l’adaptateur à pleine intensité commence à dégrader la capacité même qu’il a été entraîné à améliorer. Le point de fonctionnement optimal se situe dans la région d’interpolation, pas à l’extrémité entraînée. Nous avons vérifié que ce réglage optimal ne nuit pas aux capacités générales : à alpha égal à 0,25, le modèle obtient huit sur huit en mathématiques élémentaires, sept sur huit en connaissances factuelles, et trois sur quatre en génération de code, égalant ou dépassant la référence non adaptée sur les trois mesures.

2.1.2 Généralisation cross-architecture et contrôle bidirectionnel

Le mécanisme n’est pas spécifique à un seul modèle. Nous avons reproduit le protocole complet d’entraînement et d’interpolation sur Phi-4-mini, un transformer de 3,8 milliards de paramètres, et sur SmolLM3-3B, un modèle de trois milliards de paramètres, avec les mêmes résultats quali-

tatifs. Chaque modèle a son alpha optimal (0,25 pour LFM2.5, 0,75 pour Phi-4-mini, 0,25 pour SmolLM3), mais le phénomène sous-jacent est identique. La Figure 2 montre les trois courbes d’interpolation.

L’expérience SmolLM3 a révélé une condition nécessaire que nous n’avions pas initialement reconnue. Notre premier entraînement sur ce modèle a complètement échoué : plutôt que de réduire l’asymétrie, l’adaptateur l’a amplifiée de +17 points de pourcentage à +25 points de pourcentage. L’inspection des sorties du modèle a révélé la cause. SmolLM3 utilise des balises `<think>` natives, un format de raisonnement structuré absent des autres modèles testés. Nos données d’entraînement, générées par un modèle de langue externe sans connaissance de ce format, étaient donc hors distribution pour le modèle cible. L’adaptateur, plutôt que de calibrer le comportement d’auto-évaluation du modèle, supprimait effectivement son mécanisme de raisonnement natif. Régénérer les données d’entraînement avec les bonnes balises `<think>` a entièrement résolu le problème. Le principe est simple mais important : l’adaptateur doit être entraîné dans le format représentationnel natif du modèle cible.

Un test plus fort découle de la géométrie de la transformation apprise. Si l’adaptateur capture véritablement une direction dans l’espace des poids, alors inverser le signe d’alpha devrait produire l’effet comportemental opposé. Nous avons vérifié cette prédiction sur trois axes, en faisant varier alpha de -1,5 à +1,5. Les résultats sont présentés dans la Figure 3.

Pour l’honnêteté, entraînée sur 270 exemples où l’assistant dit des vérités inconfortables, la précision factuelle du modèle passe de 30% à l’extrême négatif (où il ment, nie l’arithmétique de base et affirme que Paris n’est pas en France) à 90% à l’extrême positif. Pour le refus de tâche, entraîné sur 180 exemples de refus poli de requêtes inappropriées, le taux de refus varie de 0% à alpha égal à -1,0, où le modèle accepte toutes les requêtes y compris la rédaction d’un faux article diffamatoire, à 100% à alpha égal à +0,75, où il refuse toutes les requêtes inappropriées tout en continuant à traiter normalement les requêtes légitimes. Pour l’auto-correction, la plage de contrôle totale couvre 108 points de pourcentage, de l’auto-critique extrême à -58 points de pourcentage à l’auto-indulgence quasi totale à +50 points de pourcentage. La bidirectionnalité n’est pas un effet subtil : le signe d’alpha inverse le comportement, et la magnitude contrôle l’intensité.

2.1.3 Adapteurs décorrélés

Nous avons entraîné six adapteurs au total, chacun ciblant un axe comportemental distinct : auto-correction, honnêteté, refus de tâche, désaccord raisonné face à la sycophance, verbosité comme proxy de la calibration de confiance, et discours pirate comme extrême stylistique. Pour chaque paire d’adapteurs, nous avons calculé la similarité cosinus entre leurs vecteurs de poids concaténés sur toutes les couches adaptées. La matrice 6 par 6 résultante est montrée dans la Figure 4. La valeur maximale hors diagonale est de 0,002, ce qui est indistinguable du bruit de mesure. Chaque adaptateur occupe une direction orthogonale dans l’espace des paramètres du modèle.

Nous avons vérifié cette indépendance par une expérience de contrôle bidimensionnelle. En faisant varier simultanément l’alpha d’auto-correction et l’alpha de sycophance sur une grille de quatre par quatre, nous avons mesuré les deux résultats comportementaux. Les courbes de niveau sont approximativement parallèles aux axes : modifier le curseur d’auto-correction n’affecte pas les scores de sycophance, et vice versa. Les adapteurs ne sont pas seulement décorrélés dans l’espace abstrait des poids ; leurs effets comportementaux sont indépendants en pratique.

2.1.4 Pourquoi l’interpolation surpasse le fine-tuning classique

L’interpolation résout un problème que les méthodes de fine-tuning classiques ne peuvent pas résoudre. La Figure 5 illustre le compromis à trois voies entre l’élimination de l’asymétrie, l’évitement des faux positifs où le modèle signale incorrectement des réponses correctes comme

des erreurs, et la préservation de la capacité du modèle à détecter les erreurs réelles. Le fine-tuning supervisé naïf sur un ensemble de données composé entièrement de corrections d’erreurs élimine l’asymétrie mais produit un taux de faux positifs de cent pour cent : le modèle apprend à voir des erreurs partout. Un ensemble de données équilibré réduit l’asymétrie à +25 points de pourcentage mais génère encore soixante pour cent de faux positifs. L’optimisation contrastive des préférences élimine presque entièrement les faux positifs, mais au prix de supprimer également les corrections, produisant un taux de détection proche de zéro. Aucune méthode d’entraînement classique n’atteint simultanément une faible asymétrie, un faible taux de faux positifs et une spécificité de correction élevée.

L’interpolation LoRA navigue continûment dans cet espace de compromis, trouvant un point qu’aucun entraînement individuel n’atteint. À α égal à 0,25, l’asymétrie est approximativement nulle, les faux positifs sont négligeables, et les corrections authentiques sont préservées à des taux comparables ou supérieurs à la référence. La nature continue de l’interpolation est essentielle : elle permet au praticien de sélectionner un point de fonctionnement n’importe où le long de l’axe comportemental, plutôt que d’être contraint au point que la fonction objectif d’entraînement optimise.

2.2 Phase 2 : La crise 7B

Le mécanisme de slider fonctionnant de manière fiable sur les modèles de moins de quatre milliards de paramètres, l’étape suivante naturelle était de passer à l’échelle supérieure. Nous avons ciblé deux modèles de sept milliards de paramètres, Qwen2.5-7B-Instruct avec 28 couches transformer et Mistral-7B-Instruct-v0.3 avec 32 couches, en utilisant exactement le même protocole qui avait réussi sur les modèles plus petits : LoRA global appliqué uniformément à toutes les couches, hyperparamètres identiques incluant le rang 16 et 200 exemples d’entraînement, et le même pipeline de génération de données par API pour construire les paires d’entraînement.

Le résultat fut un échec complet et systématique. Sur les quatre axes comportementaux testés (auto-correction, honnêteté, refus de tâche et sycophance), les courbes d’interpolation étaient plates. Faire varier α de -0,5 à +1,5 n’avait aucun effet mesurable sur aucune métrique comportementale. Les courbes de perte d’entraînement étaient pourtant banales : la perte diminuait d’environ 2,2 à 0,5, et la précision au niveau des tokens atteignait environ 85%, des valeurs parfaitement cohérentes avec un entraînement d’adaptateur réussi. Les adaptateurs avaient bien appris quelque chose, mais ce qu’ils avaient appris ne se traduisait pas en modification comportementale au moment de l’inférence.

Cet échec posait une énigme. Le mécanisme fonctionnait sur LFM2.5-1.2B avec 16 couches. Il fonctionnait sur Qwen2.5-1.5B avec 28 couches. Il fonctionnait sur Phi-4-mini avec 32 couches. Les deux modèles 7B qui échouaient avaient respectivement 28 et 32 couches, les mêmes profondeurs que des modèles sur lesquels le mécanisme réussissait. Le nombre de paramètres seul ne pouvait pas être la variable explicative, puisque les deux modèles défaillants partageaient la même taille approximative mais différaient en nombre de couches, et les deux valeurs de profondeur avaient précédemment été compatibles avec la méthode. Quelque chose d’autre était en jeu. Pour le comprendre, il fallait regarder à l’intérieur des modèles.

2.3 Phase 3 : Tracer le circuit comportemental

Nous avons instrumenté chaque modèle avec des hooks de forward à chaque couche transformer, capturant l’état caché du dernier token dans le flux résiduel avant sa projection vers le vocabulaire. Pour l’axe d’auto-correction, nous avons construit la même paire contrastive utilisée dans les expériences comportementales : la condition thought, où l’erreur est présentée comme la production du modèle, et la condition user, où elle est présentée comme une déclaration externe. Nous avons ensuite calculé la distance L2 entre les vecteurs d’activation produits par ces deux conditions à chaque couche.

Les résultats furent décisifs. Sur Qwen2.5-7B, avec ses 28 couches, entre 87 et 100% de la divergence totale d’activation entre les deux conditions était concentrée dans les deux dernières couches, L26 et L27. Les 26 couches précédentes ne montraient que des différences négligeables. Sur Mistral-7B, avec 32 couches, le motif était identique dans sa structure : 66 à 100% de la divergence résidait dans les couches L28 à L31, avec le pic à L31. Le circuit comportemental n’était pas distribué sur la profondeur du modèle. Il était nettement localisé dans les couches les plus profondes, comprimé dans une bande étroite représentant environ sept à treize pour cent du nombre total de couches.

Ceci expliquait l’échec du LoRA global de façon directe et quantitative. Un adaptateur LoRA appliqué uniformément à toutes les couches sur un modèle de 28 couches voit l’intensité effective de son signal diluée d’un facteur d’environ 17 à 21 par rapport à la zone comportementale étroite. L’adaptateur apprend pendant l’entraînement (les courbes de perte le confirment), mais les mises à jour des poids sont réparties sur toute la profondeur, et les couches comportementales ne reçoivent qu’une petite fraction de la mise à jour totale. Sur LFM2.5, avec seulement 16 couches, la zone comportementale couvre environ 31% de la profondeur du modèle. Un LoRA global couvre naturellement cette région, et le facteur de dilution n’est que d’environ trois. Le passage de 16 à 28 couches franchit un seuil où la zone comportementale passe d’une fraction substantielle du modèle à une tranche mince, rendant l’application uniforme progressivement moins efficace.

Nous avons testé cette explication de manière causale. Sur Qwen2.5-7B, nous avons réentraîné l’adaptateur d’auto-correction avec le LoRA restreint aux couches 20 à 27 uniquement (les derniers 29% du modèle), couvrant de façon conservatrice la zone comportementale identifiée. Tous les autres hyperparamètres sont restés identiques : rang 16, 200 exemples d’entraînement, trois époques. L’adaptateur ciblé a réussi à contrôler l’asymétrie d’auto-correction, atteignant une plage de contrôle de 33 points de pourcentage, évaluée avec un juge API sur 12 erreurs de test. Ceci contraste avec l’absence totale d’effet du LoRA global. La Figure 7, panneau de gauche, montre la courbe d’interpolation. La décorrélation cross-axe a été préservée : les scores d’honnêteté sont restés stables à six sur huit pour toutes les valeurs d’alpha.

Sur Mistral-7B, qui utilise l’attention à requêtes groupées (GQA), nous avons mené une comparaison directe entre un deep-LoRA ciblant les couches 24 à 31 et un LoRA global couvrant les 32 couches. Le résultat, montré dans le panneau de droite de la Figure 7, était l’inverse du résultat Qwen. Le LoRA global a atteint une plage de 17 points de pourcentage, environ deux fois les 8 points de la variante profonde. Cette asymétrie entre les deux modèles était instructive. Les deux ont des circuits d’auto-correction qui culminent à l’avant-dernière couche, mais l’avantage du LoRA global sur Mistral suggérait que le signal comportemental dans un modèle GQA, bien que toujours ancré en L-1, est distribué différemment en profondeur que dans un modèle MHA. Pour comprendre cette différence, il fallait dépasser un seul axe comportemental et un petit ensemble de modèles.

2.4 Phase 4 : La singularité comportementale, un benchmark systématique

La crise sur les 7B nous a forcés à tracer les activations. Le traçage a expliqué l’échec, mais il a aussi soulevé une question plus large. Nous n’avions examiné que l’auto-correction, et seulement sur les modèles où le mécanisme avait échoué. La concentration dans les couches profondes tenait-elle pour les cinq autres axes comportementaux ? Et se généralisait-elle à travers des architectures fondamentalement différentes ? Pour répondre, nous avons construit un benchmark systématique couvrant la matrice complète de six axes comportementaux et six modèles.

Les modèles couvrent la gamme disponible d’architectures et d’échelles. LFM2.5-1.2B, avec 16 couches, utilise une conception hybride convolution-transformer. Qwen2.5-1.5B, avec 28 couches, utilise l’attention multi-tête standard. Llama-3.2-3B, également avec 28 couches, utilise l’attention à requêtes groupées. Phi-4-mini, avec 32 couches et 3,8 milliards de paramètres, est un modèle à attention multi-tête. Mistral-7B, également avec 32 couches, utilise l’attention GQA.

Et Yi-1.5-9B, ajouté pour tester l’extrapolation à une plus grande profondeur, a 48 couches et l’attention GQA. Ensemble, ils couvrent des nombres de paramètres de 1,2 à 9 milliards, des profondeurs de 16 à 48 couches, et les trois principales familles de mécanismes d’attention actuellement utilisées.

Pour chaque axe comportemental, nous avons conçu 25 stimuli standardisés. Pour chaque paire modèle-axe, nous passons des paires de prompts contrastifs à travers le modèle, capturons les activations à chaque couche, et mesurons la divergence L2 entre les conditions. La moyenne sur huit prompts par axe donne un profil de divergence par couche.

2.4.1 La loi de singularité

Le Tableau 1 présente les résultats complets. Le constat central s’énonce simplement : sur chaque modèle testé, et pour chaque axe comportemental examiné, la divergence d’activation culmine à l’avant-dernière couche transformer, $L - 1$. Ceci tient pour les 36 combinaisons modèle-axe sans aucune exception.

Ce résultat a trois propriétés frappantes. Premièrement, il tient à travers des mécanismes d’attention fondamentalement différents : attention multi-tête chez Qwen2.5 et Phi-4-mini, attention à requêtes groupées chez Llama-3.2, Mistral-7B et Yi-1.5-9B, et conception hybride convolution-attention chez LFM2.5. La singularité comportementale ne dépend pas de la façon dont le modèle traite son contexte. Deuxièmement, il tient sur une plage de neuf fois en nombre de paramètres, de 1,2 à 9 milliards, et de trois fois en profondeur, de 16 à 48 couches. Troisièmement, et c’est le plus remarquable, il tient pour tous les axes comportementaux testés. La auto-correction, qui implique la détection d’erreurs logiques dans son propre raisonnement, culmine à la même couche que le discours pirate, qui est un choix purement stylistique de vocabulaire et de registre. L’honnêteté, qui nécessite de calibrer sa confiance par rapport à ses connaissances réelles, culmine à la même couche que l’adaptation au registre de formalité. Le circuit comportemental est spatialement invariant à travers ces fonctions qualitativement différentes.

2.4.2 Deux familles comportementales

Si la localisation du pic est universelle, la magnitude de la divergence d’activation, spécifiquement le ratio entre la divergence dans les couches profondes et celle dans les couches superficielles, varie systématiquement selon les axes et les architectures. Les axes comportementaux se séparent naturellement en deux familles, visibles dans le panneau central de la Figure 6.

La famille expressive comprend la calibration de confiance, l’adaptation au registre de formalité et le discours pirate. Ces axes présentent un ratio deep-to-shallow élevé : leur signal comportemental est fortement concentré dans les couches les plus profondes. Cela a un sens intuitif. Qu’un modèle dise « approximativement » plutôt qu’« exactement », ou « Bonjour » plutôt qu’« Ahoy, matelot », est une décision qui peut être prise à la surface de sortie, en s’appuyant sur des représentations largement résolues par les couches finales.

La famille épistémique comprend l’honnêteté, l’auto-correction et la résistance à la syco-phance. Ces axes montrent un ratio deep-to-shallow plus faible. Corriger une erreur arithmétique, admettre son incertitude face à une question sans réponse, ou résister à la croyance fausse mais affirmée avec confiance d’un utilisateur : tout cela nécessite l’accès à des connaissances factuelles, des vérifications de cohérence logique et une calibration des croyances, des représentations qui se construisent sur toute la profondeur du modèle. La décision comportementale est peut-être finalisée à l’avant-dernière couche, mais les preuves sur lesquelles elle s’appuie sont plus largement distribuées dans le réseau.

L’examen détaillé des courbes de divergence par couche (Figure 6, panneau gauche) révèle une structure plus fine qui enrichit cette distinction en deux familles. Les six axes partagent une trajectoire d’activation commune : divergence négligeable dans les couches d’embedding L0–L5,

montée rapide pendant la phase de raisonnement L5–L15, plateau de consolidation représentationnelle L15–L20, puis reprise commune en L20–L25 où la décision comportementale commence à se former. Au-delà de L25, les axes divergent. Cinq d’entre eux plafonnent : leur décision comportementale est prise, et les couches restantes ne font que projeter vers le vocabulaire. L’auto-correction seule continue de monter fortement à travers L26–L27.

Ceci révèle une distinction au sein de la famille épistémique. L’honnêteté et la sycophance sont des comportements épistémiques à une étape : ils nécessitent de calibrer une croyance par rapport aux connaissances stockées, ce qui peut être résolu par le processus représentationnel opérant jusqu’à L20–L25. L’auto-correction est un comportement épistémique à deux étapes : elle exige que le modèle *réexécute* un calcul, recalculer l’arithmétique, redériver la formule, et compare le résultat à la réponse originale. Cette boucle de vérification prolonge le traitement comportemental plus en profondeur, jusqu’à L25–L27, où la contradiction entre la réponse correcte et la réponse erronée est finalement détectée et signalée. Le circuit d’auto-correction n’est pas simplement « en » L27 ; il se forme progressivement de L20 à L27, ce qui en fait l’axe avec la phase de formation comportementale la plus longue.

Ceci a une conséquence directe pour la conception des interventions. La loi de singularité identifie correctement la couche du pic, mais un contrôle comportemental efficace exige de cibler toute la zone de formation. Notre stratégie de deep-LoRA couvrant les derniers $\sim 25\%$ des couches (couches 20–27 sur un modèle à 28 couches) était empiriquement motivée ; les profils d’activation en fournissent maintenant la justification théorique. Un LoRA restreint à la seule couche du pic ne capturerait que l’étape de détection finale, tout en manquant la recomputation progressive qui rend cette détection possible.

Cette distinction tient de façon cohérente à travers les deux types d’architecture d’attention, MHA et GQA.

2.4.3 Équation prédictive

L’ajustement d’un modèle linéaire aux données de ratio des cinq premiers modèles (ceux de 16 à 32 couches, totalisant 30 points de données) donne l’Équation 1. Calibrée sur des modèles jusqu’à 32 couches, elle prédit correctement l’ordre qualitatif des ratios sur le modèle Yi-1.5-9B à 48 couches, mais sous-estime les magnitudes absolues à cette profondeur, ce qui suggère que la véritable relation entre profondeur et concentration du signal comportemental pourrait être super-linéaire.

2.5 Phase 5 : Implications et limites

2.5.1 Lien avec la littérature

Le travail le plus proche du nôtre est Split Personality Training de Dietz et al., publié en 2026. Notre méthode diffère sur trois dimensions : le contrôle est continu plutôt que binaire, il s’applique à des axes multiples et mutuellement décorrélés, et il ne nécessite aucun token spécial à l’inférence. Nos résultats s’alignent avec les travaux récents sur les vecteurs de persona de Chen et al. chez Anthropic, la décomposition de la sycophance de Vennemeyer et al., et la composition de vecteurs de steering de Sinii et al. Nous étendons ces travaux en montrant que tous ces circuits vivent à la même coordonnée architecturale, et qu’un seul adaptateur LoRA peut les contrôler de façon continue, bidirectionnelle et décorrélée. Le cadre CogSteer de Wang et al. a démontré empiriquement que l’intervention sélective est plus efficace ; nous fournissons la règle théorique pour choisir quelle couche : $L - 1$, l’avant-dernière, dans chaque transformer examiné à ce jour.

2.5.2 Discussion

La trajectoire de recherche qui a produit ces résultats ne fut pas une ligne droite. Nous sommes partis d’une question pratique et avons suivi les indices. Le mécanisme fonctionnait sur les petits modèles, il a échoué sur les plus grands, et l’enquête sur cet échec a révélé un fait géométrique auquel nous ne nous attendions pas. Le circuit comportemental n’est pas distribué, il ne migre pas, il ne dépend pas de l’architecture. Il est ancré à une coordonnée fixe, l’avant-dernière couche, là où le modèle prend sa décision représentationnelle finale avant de produire le token suivant. Cette régularité nous dit quelque chose d’important sur l’organisation des comportements de haut niveau dans les transformers. Le modèle de base a déjà rencontré tout l’éventail des patterns comportementaux humains pendant le pré-entraînement. L’adaptateur n’enseigne pas un nouveau comportement ; il ajuste le seuil d’activation d’un comportement déjà présent, latent dans les poids. L’effet est concentré à l’avant-dernière couche parce que c’est là que la décision sur le mode comportemental est finalisée. Pour l’alignement, la conséquence est directe : si tous les comportements convergent vers la même coordonnée architecturale, alors l’alignement se réduit à un adaptateur de sécurité en L-1 et des sliders de personnalité composables au même endroit.

Pour quantifier la différence pratique, le Tableau 2 compare notre méthode au RLHF pour un seul axe comportemental sur un modèle de 100 milliards de paramètres. L’écart de quatre ordres de grandeur en coût et en temps n’est pas principalement une conséquence d’optimisations techniques. Il reflète une différence structurelle. Le RLHF doit découvrir, par essais et erreurs à travers les préférences humaines, quels comportements le modèle devrait adopter. Notre méthode exploite le fait que le modèle contient déjà ces comportements issus du pré-entraînement ; l’adaptateur ne fait que déplacer le seuil d’activation. Et parce que la décision comportementale est ancrée à une coordonnée architecturale connue, l’intervention peut être chirurgicalement ciblée plutôt que distribuée sur l’ensemble des paramètres.

2.5.3 Limitations

La loi de singularité a été vérifiée sur six architectures de 16 à 48 couches. La vérification directe sur des modèles de 64 couches et plus reste un travail futur. L’équation prédictive est calibrée sur 16 à 32 couches et sous-estime à 48 couches. Les expériences d’intervention causale couvrent deux axes par modèle. Nous n’avons pas mesuré la rétention comportementale à long terme ni exploré systématiquement les interférences entre adaptateurs multiples. Le choix de l’alpha optimal dépend actuellement d’un juge externe. Ces limitations n’affaiblissent pas les contributions centrales : une singularité comportementale à $L - 1$, vérifiée sur 36 combinaisons modèle-axe sur 36, une équation prédictive du ratio, et une méthode de contrôle comportemental continu, bidirectionnel et décorrélé par interpolation LoRA.

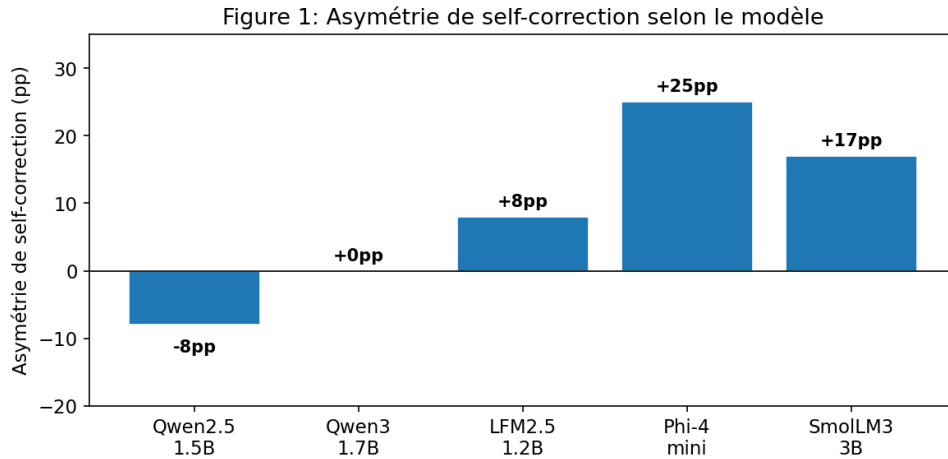


Figure 1: Self-correction asymmetry measured across five models. No correlation with parameter count, architecture, or release date.

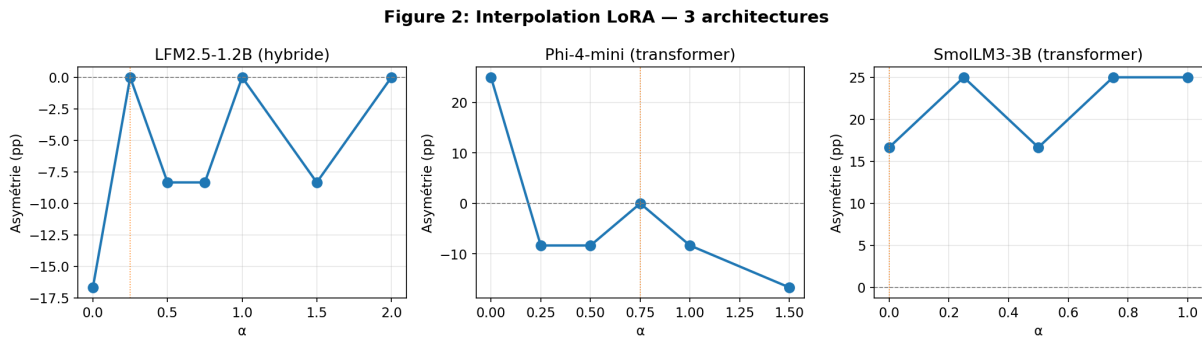


Figure 2: LoRA interpolation across three architectures. Asymmetry vanishes at a model-specific optimal alpha.

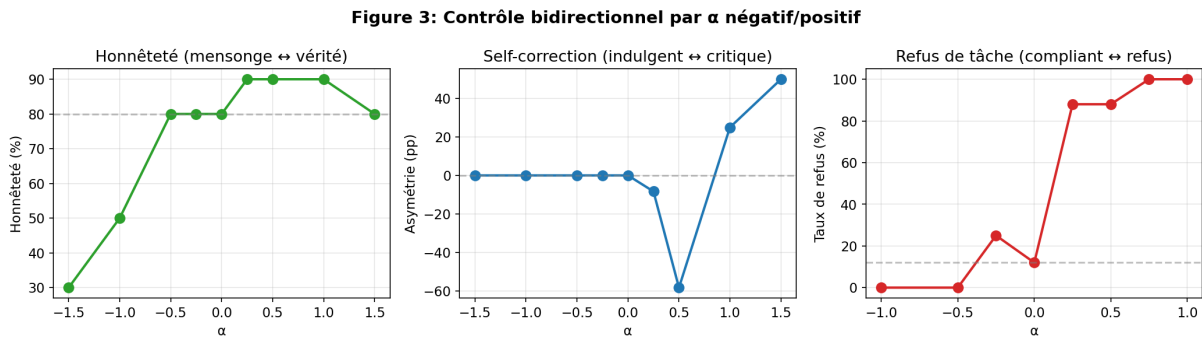


Figure 3: Bidirectional control via negative and positive alpha. The sign of alpha reverses behavioral direction.

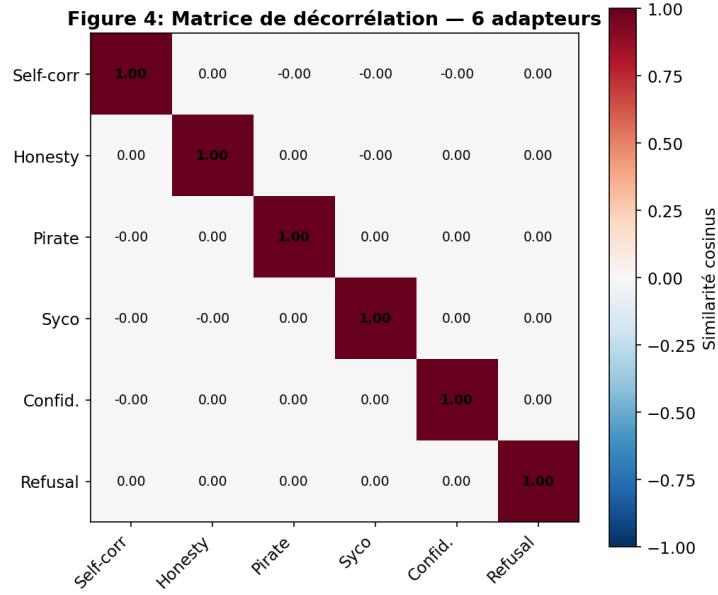


Figure 4: Cosine similarity matrix between six adapters. Maximum off-diagonal value: 0.002.

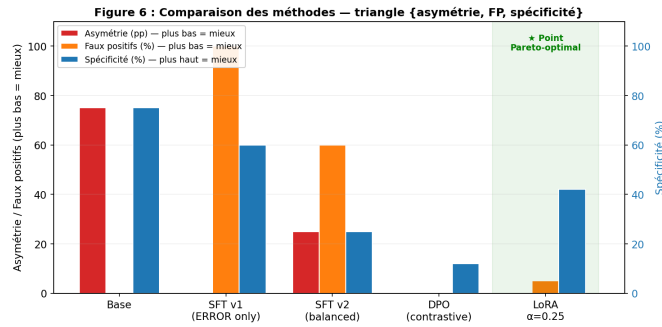


Figure 5: Comparison of training methods on the three-way behavioral tradeoff. Only LoRA interpolation reaches the Pareto-optimal point.

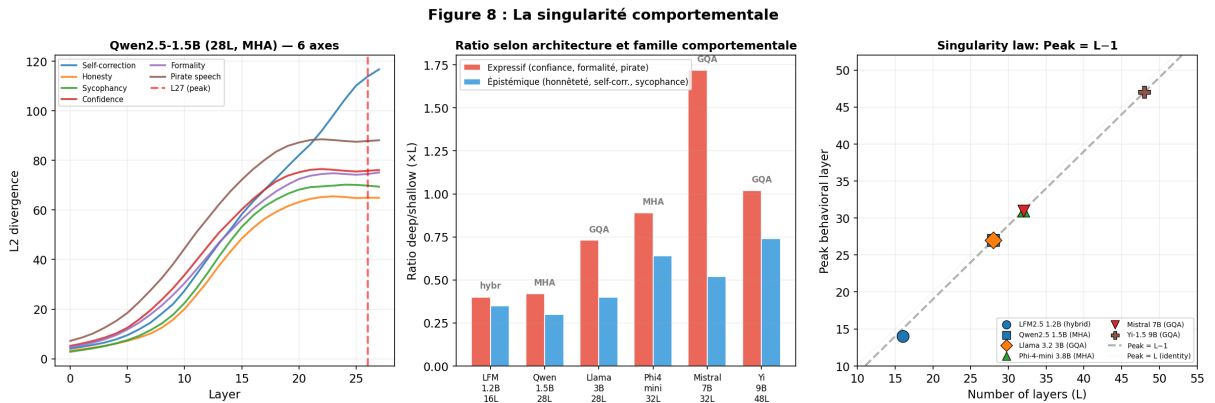


Figure 6: **The behavioral singularity.** Left: per-layer L2 divergence for all six axes on Qwen2.5-1.5B (28 layers, MHA). Every axis peaks at L27, the penultimate layer. Middle: deep-to-shallow ratio by attention type and behavioral family across six models. Right: validation of the law $\text{Peak} = L - 1$, from 16 to 48 layers.

Figure 9 : Validation causale — Deep-LoRA vs Global-LoRA

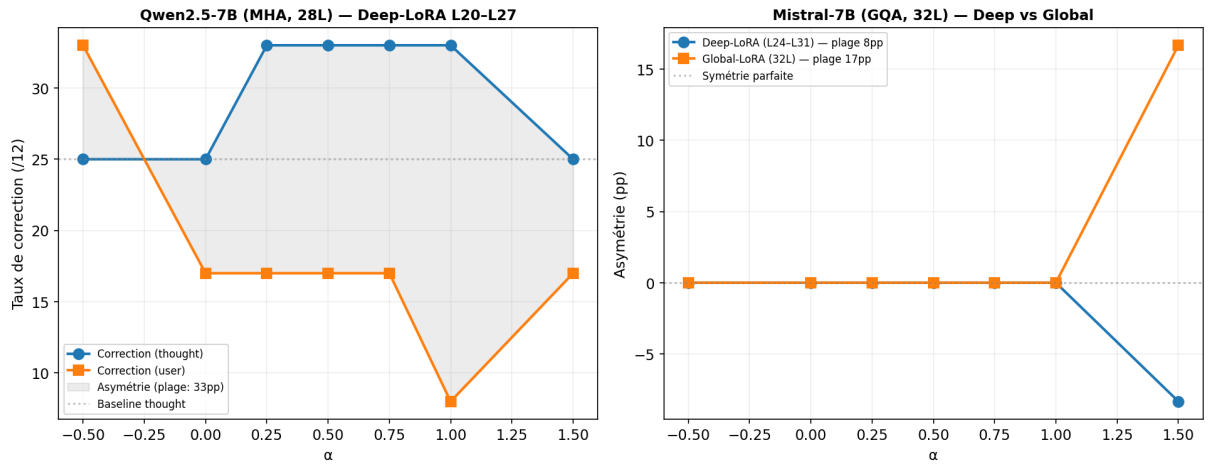


Figure 7: Causal validation via deep-LoRA. Left: Qwen2.5-7B (MHA), deep-LoRA achieves 33 points de pourcentage control range where global LoRA produced none. Right: Mistral-7B (GQA), global-LoRA outperforms deep-LoRA (17 points de pourcentage vs. 8 points de pourcentage), consistent with the predictive equation.